GROUP ASSIGNMENT: LEETCODE PROBLEM 203

Problem: Remove Linked List Elements

Programming language: Java

Course: csc 301

Department: Software Engineering

Group leader: NTAH CHIDERA ISAAC

Group Members Names and Matric Numbers including the Leader

1. GBIRISON IYIMIDE GABRIEL — VUG/SEN/24/12154
2. OGBECHE ALEXIS ININUNG — VUG/SEN/24/12266
3. CHIDI KAMSY — VUG/SEN/24/12288
4. NWABUA JOSEPH CHIAGOZIEM — VUG/SEN/24/12377
5. MORAKINYO SHAMMAH MUYIWA — VUG/SEN/24/12779
6. NTAH CHIDERA ISAAC — VUG/SEN/24/11663
7. OLUGBAMI OLABODE — VUG/SEN/24/12935
8. OMOKHAGBOR SAMUEL OSOMINESHI — VUG/SEN/24/14562
9. OLADEJI ADISA — VUG/SEN/23/10448
10. UDE-EZE CHIGOZIE DANIEL — VUG/SEN/24/11852

1. Introduction

A linked list is a linear data structure consisting of nodes. Each node stores a value and a reference to the next node. This assignment solves LeetCode Problem 203, Remove Linked List Elements, using Java.

2. Problem statement

Given the head of a singly linked list and an integer val, remove every node whose value equals val and return the head of the modified list.

Example input: list = 1 → 2 → 6 → 3 → 6; val = 6

Expected output: 1 → 2 → 3

Explanation: Both nodes containing 6 are removed. The remaining nodes retain their original order

4. Algorithm

1. Create a dummy node and connect its next reference to the original head.
2. Set current to the dummy node.
3. While current.next exists, examine the next node.
4. If current.next.val equals val, skip that node by connecting current.next to current.next.next.
5. Otherwise, advance current to current.next.
6. Return dummy.next as the head of the modified list.

5. Java source code

```
class Solution {
    public ListNode removeElements(ListNode head, int val) {
        ListNode dummy = new ListNode(0);
        dummy.next = head;

        ListNode current = dummy;

        while (current.next != null) {
            if (current.next.val == val) {
                current.next = current.next.next;
            } else {
                current = current.next;
            }
        }

        return dummy.next;
    }
}
```

6. Code explanation

Dummy node: A node placed before the original head so removal of the first node is handled consistently.

dummy.next = head: Connects the dummy node to the original list.

current: A reference used to move through the list.

while (current.next != null): Repeats while there is a next node to examine.

if (current.next.val == val): Checks whether the next node contains the target value.

current.next = current.next.next: Skips the next node when it should be removed.

current = current.next: Moves forward when the next node should remain.

return dummy.next: Returns the head of the updated list.

Diagram Example

BEFORE (target value = 6):

head → [1] → [2] → [6] → [3] → [6] → null

AFTER removing every node with value 6:

head → [1] → [2] → [3] → null